

Numerical Series & Patterns: 20 More Practice Questions

Medium to Hard — With Hints

A further 20 questions on numerical series and patterns, all solvable using loops and conditional statements. Ordered medium → hard, with hints scaled to difficulty.

Q1. Strong Number Checker (Medium)

A “strong number” equals the sum of the factorials of its own digits (e.g., $145 = 1! + 4! + 5!$). Check whether a given number is a strong number.

Hint: You’ll need a way to compute a digit’s factorial — either a small loop inside your digit loop, or a helper function — since Python has no built-in single-digit factorial shortcut without importing `math.factorial`.

Q2. Automorphic Number Checker (Medium)

A number is “automorphic” if its square ends with the number itself (e.g., $25^2 = 625$, which ends in `25`). Print all automorphic numbers up to `n`.

Hint: Compare the *last few digits* of the square to the original number — converting both to strings and using slicing (`str(square)[-len(str(num)):]`) is often easier than doing this with pure modulo arithmetic.

Q3. Disarium Number Checker (Medium)

A number is a “Disarium” number if the sum of its digits, each raised to the power of its *position* (1st digit to the power 1, 2nd digit to the power 2, etc.), equals the number itself (e.g., $135 = 1^1 + 3^2 + 5^3$). Check if a given number is Disarium.

Hint: Loop through the digits using `enumerate()` (starting the position count at 1, not 0) so each digit knows its own position to use as the exponent.

Q4. Spy Number Checker (Medium)

A number is a “Spy number” if the sum of its digits equals the product of its digits (e.g., `1124`: sum = 8, product = 8). Check if a given number is a Spy number.

Hint: You’ll need two separate accumulators in the same loop — one starting at `0` for the sum, and one starting at `1` (not `0`!) for the product, since multiplying by zero would wipe out the result.

Q5. Tech Number Checker (Medium/Hard)

A “Tech number” has an even number of digits; when split into two equal halves, the square of the sum of these two halves equals the original number (e.g., `81`: halves are `8` and `1`, sum = `9`, and $9^2 = 81$). Print all Tech numbers up to `n`.

Hint: First check that the number has an even digit count (odd-digit numbers can never qualify) — then split the digit string exactly in half using slicing, convert each half back to an integer, and test the condition.

Q6. Niven (Harshad) Number Checker (Medium)

A number is a Niven (or Harshad) number if it is divisible by the sum of its own digits (e.g., `18 : digit sum = 9`, and `18 % 9 == 0`). Print all Niven numbers in a given range.

Hint: Compute the digit sum first with a small loop, then check divisibility — watch out for the digit sum being `0` only if the number itself is `0` (division by zero), which shouldn't occur for a properly bounded positive range, but is worth thinking through.

Q7. Tribonacci Series (Medium)

Print the first `n` terms of the Tribonacci sequence, where each term is the sum of the *three* preceding terms (starting `0, 1, 1`).

Hint: You'll need three "running" variables instead of Fibonacci's two — think carefully about how all three should shift forward together in a single line, similar to Fibonacci's `a, b = b, a + b` trick, but extended.

Q8. Sum of the Series $1/1! + 2/2! + 3/3! + \dots + N/N!$ (Medium/Hard)

Compute the sum of this series up to `n` terms.

Hint: You don't need to recompute each factorial from scratch every iteration — since `k! = k × (k-1)!`, you can keep a running factorial variable that updates incrementally across loop iterations, rather than calling a factorial function repeatedly.

Q9. Harmonic Series Sum (Medium)

Compute the sum `1 + 1/2 + 1/3 + ... + 1/n` for a given `n`.

Hint: Make sure you're using floating-point division (`/`, not `//`) — a very common mistake here is accidentally using integer division and getting `0` for every term after the first.

Q10. Recamán's Sequence (Hard)

Generate the first `n` terms of Recamán's sequence: starting at `a(0) = 0`, each next term is `a(n-1) - n` if that value is positive **and hasn't been used before**; otherwise, it's `a(n-1) + n`.

Hint: You'll need a set to track every value that's already appeared in the sequence so far, since the "hasn't been used before" condition can't be checked just by looking at the previous term alone.

Q11. Duck Numbers in a Range (Medium)

A "Duck number" contains at least one zero digit, but does **not** start with zero (e.g., `102` is a Duck number, but `087` isn't a valid number to begin with, and `1002` is). Print all Duck numbers in a given range.

Hint: Since Python integers never have leading zeros anyway, the real check is just "does `'0'` appear anywhere in the string form of the number *after* the first character?" — think about whether checking the whole string vs. just `string[1:]` for a zero actually matters here.

Q12. Neon Numbers in a Range (Medium)

A number is "Neon" if the sum of the digits of its square equals the number itself (e.g., `9 : 92 = 81`, and `8 + 1 = 9`). Print all Neon numbers in a given range.

Hint: This is a two-step digit process per number: first square it, then sum the digits of *that* result — don't confuse this with summing the digits of the original number.

Q13. Ugly Numbers (Hard)

“Ugly numbers” are positive numbers whose only prime factors are 2, 3, or 5 (e.g., 1, 2, 3, 4, 5, 6, 8, 9, 10, 12... — note that 7, 11, 14 are NOT ugly since they involve other prime factors). Find the n th ugly number.

Hint 1: Checking every number one by one and testing “is it ugly?” by repeatedly dividing out 2s, 3s, and 5s (checking if you’re left with 1) works but can be slow for large n — still a valid loop-based approach to start with.

Hint 2: For a given candidate number, repeatedly divide it by 2 as long as it’s evenly divisible, then by 3, then by 5 — if what remains after all that is exactly 1, it was ugly. Keep a counter of how many ugly numbers you’ve found so far until you reach the n th one.

Q14. Prime Palindromes in a Range (Medium/Hard)

Print all numbers in a given range that are **both** prime and palindromic (e.g., 131, 757).

Hint: Write (or reuse) two separate checks — one for primality (loop up to the square root), one for being a palindrome (compare the string to its reverse) — and only print numbers that pass both.

Q15. Catalan Numbers (Hard)

Compute the n th Catalan number using the recurrence $C(0) = 1$ and $C(n) = C(n-1) * 2 * (2n - 1) / (n + 1)$.

Hint: Use a loop that builds up from $C(0)$ to $C(n)$ iteratively (rather than computing it from scratch with factorials each time), keeping a single running variable that gets updated using the recurrence formula at each step — be careful to use integer division correctly, since the formula is guaranteed to produce a whole number at every step even though it involves division.

Q16. Steps to Reduce a Number to 1 (Medium/Hard)

Given a number, find the minimum number of steps to reduce it to 1 using the rule: if even, divide by 2; if odd, either add 1 or subtract 1 (choose whichever leads to fewer total steps).

Hint: For odd numbers, think about which choice (add 1 or subtract 1) leads to an even number that can then be divided repeatedly — a greedy rule based on checking the number modulo 4 handles almost every case correctly, but make sure to special-case the number 3 (where subtracting is better) and 1 (already done, zero steps needed).

Q17. Number Diamond Pattern (Using Digits, Not Stars) (Medium)

Print a diamond pattern using numbers instead of *, where each row’s numbers count up to the row number and back down:

```
1
121
12321
1234321
```

Hint: Combine the spacing logic from a star-diamond pattern with the “count up then down” numeric row logic from Floyd/Pascal-style triangles — build each row as a string first (numbers + reversed numbers, minus the duplicated peak), then center or left-pad it with the right number of spaces.

Q18. Alternating Harmonic-Like Series: $1 - 1/2 + 1/3 - 1/4 + \dots$ (Medium/Hard)

Compute the sum of this alternating series up to n terms (this series famously converges toward $\ln(2)$ as n grows very large).

Hint: Combine the alternating-sign logic (Q9 from the earlier solved set) with proper floating-point division (Q9 from this set) — decide whether to use an `if-else` on `i % 2` or a `(-1) ** (i+1)` multiplier to control the sign each iteration.

Q19. Emirp Numbers in a Range (Hard)

An “Emirp” (prime spelled backwards) is a prime number whose digit-reversal is a **different** prime number (e.g., 13 is Emirp since it’s prime and its reverse, 31, is also prime and different from 13; but a palindrome prime like 11 doesn’t count, since its reverse is the same number). Print all Emirp numbers in a given range.

Hint: For each candidate number, run three checks in sequence: (1) is it prime? (2) reverse its digits — is the reversed number also prime? (3) is the reversed number actually *different* from the original? All three must hold.

Q20. Digit Sum Persistence (Additive Persistence) (Medium/Hard)

Given a number, repeatedly replace it with the sum of its digits until a single digit remains. Print both the final single digit AND how many rounds of summing it took to get there (this “number of rounds” is called the number’s “additive persistence”).

Hint: This needs a `while` loop that keeps going as long as the current value has more than one digit (check with `len(str(current)) > 1`), with a counter incremented once per full digit-summing pass — don’t confuse “one pass of summing all digits” with “one loop iteration over individual digits,” since each pass itself needs its own inner loop or generator expression.

Try solving these before checking any reference solutions — each one builds directly on the loop/conditional patterns from the earlier 15 solved problems (running accumulators, digit extraction, nested loops for pattern printing, and while-loop-driven iterative processes).